



中国海洋大学
信息科学与工程学部

UDP 与 TCP 协议分析

课程名称 计算机网络

指导教师 洪峰

学生姓名 刘和雄

学号 24070001071

专业/班级 24 软件工程

学年学期 2025 秋季学期

提交日期 2026 年 1 月 2 日



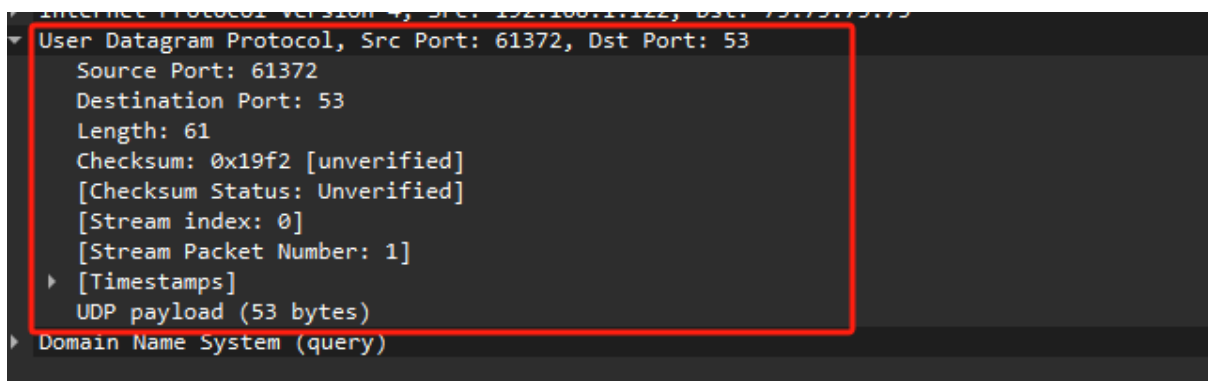
目录

1	UDP 协议分析	2
1.1	UDP 消息结构观察	2
1.2	问题回答	2
2	TCP 协议分析	3
2.1	TCP 连接建立 (三次握手)	3
2.2	TCP 连接断开 (Teardown)	5
2.3	TCP 性能图表分析	5
3	实验总结	7

1 UDP 协议分析

1.1 UDP 消息结构观察

操作步骤：打开 trace-udp.pcap，在过滤器输入 udp，选中任意一个数据包并观察中间详情面板。



(这里展示的是一个 DNS 查询包)

图片数据解读：根据截图观察，该 UDP 数据包的具体信息如下：

- Source Port: 61372
- Destination Port: 53 (DNS 服务端口)
- Length: 61 (Header 8 字节 + Payload 53 字节)
- Checksum: 0x19f2

1.2 问题回答

1.2.1 Q1: Length 字段包含什么内容？

Length 字段包含 UDP 头部 (Header) 和 UDP 数据载荷 (Payload) 的总长度。

1.2.2 Q2: UDP Checksum 的长度是多少位 (bits)？

16 bits (即 2 字节)。



1.2.3 Q3: 整个 UDP Header 的长度是多少字节 (bytes)?

8 bytes。UDP 头部包含 Source Port (2 字节) + Destination Port (2 字节) + Length (2 字节) + Checksum (2 字节) = 8 字节。

1.2.4 Q4: 在 IP 头部中, 用于标识上层协议为 UDP 的 Protocol 字段值是多少?

17。IP 头部中的 Protocol 字段值 17 代表 UDP 协议, 而 6 代表 TCP 协议。

1.2.5 Q5: 目的 IP 地址分析

观察 Trace 文件, 除了本机和特定服务器外, 你还看到了哪些其他的目的 IP 地址 (例如广播或多播地址)? 在 Trace 中可以观察到多播 (Multicast) 和广播 (Broadcast) 地址。例如用于 SSDP (简单服务发现协议) 的地址 239.255.255.250, 或者用于 MDNS 的多播地址。

1.2.6 Q6: 你的 Trace 文件中, 典型的 UDP 消息大小是多少?

大多数 UDP 消息都比较短, 远小于 64KB 的最大值。如截图所示, DNS 查询包长度仅为 61 字节。Trace 中其他常见值通常在 86, 95, 175 等小数值范围内。

1.2.7 UDP 消息结构草图

Source Port (2 bytes) → Destination Port (2 bytes) → Length (2 bytes) → Checksum (2 bytes) → Payload (Variable)

2 TCP 协议分析

2.1 TCP 连接建立 (三次握手)

操作步骤: 打开 trace-tcp.pcap, 寻找标志位包含 SYN 的数据包。



```
21 0 000068 192.168.1.122 64 238 147 113 TCP 66 60643 → 8
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1011 .... = Header Length: 44 bytes (11)
Flags: 0x002 (SYN)
000. .... = Reserved: Not set
...0 .... = Accurate ECN: Not set
.... 0... = Congestion Window Reduced: Not set
.... .0.. = ECN-Echo: Not set
.... ..0. = Urgent: Not set
.... ...0 = Acknowledgment: Not set
.... ....0... = Push: Not set
.... .... .0.. = Reset: Not set
.... .... ..1. = Syn: Set
.... .... ...0 = Fin: Not set
[TCP Flags: .....S.]
Window: 65535
[Calculated window size: 65535]
Checksum: 0x2212 [unverified]
```

```
41 0 000068 192.168.1.122 64 238 147 113 TCP 66 60643 → 8
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 2682012318
1010 .... = Header Length: 40 bytes (10)
Flags: 0x012 (SYN, ACK)
000. .... = Reserved: Not set
...0 .... = Accurate ECN: Not set
.... 0... = Congestion Window Reduced: Not set
.... .0.. = ECN-Echo: Not set
.... ..0. = Urgent: Not set
.... ...1 = Acknowledgment: Set
.... ....0... = Push: Not set
.... .... .0.. = Reset: Not set
.... .... ..1. = Syn: Set
.... .... ...0 = Fin: Not set
[TCP Flags: .....A..S.]
Window: 5792
[Calculated window size: 5792]
Checksum: 0x67d7 [unverified]
```

```
61 0 000068 192.168.1.122 64 238 147 113 TCP 66 60643 → 8
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 349487777
1000 .... = Header Length: 32 bytes (8)
Flags: 0x010 (ACK)
000. .... = Reserved: Not set
...0 .... = Accurate ECN: Not set
.... 0... = Congestion Window Reduced: Not set
.... .0.. = ECN-Echo: Not set
.... ..0. = Urgent: Not set
.... ...1 = Acknowledgment: Set
.... ....0... = Push: Not set
.... .... .0.. = Reset: Not set
.... .... ..0. = Syn: Not set
.... .... ...0 = Fin: Not set
[TCP Flags: .....A....]
Window: 65535
[Calculated window size: 524280]
[Window size scaling factor: 8]
```

[显示 TCP 三次握手的前三个数据包]

时序分析：根据截图中的 Info 列和 Flags 详情，三次握手过程如下：

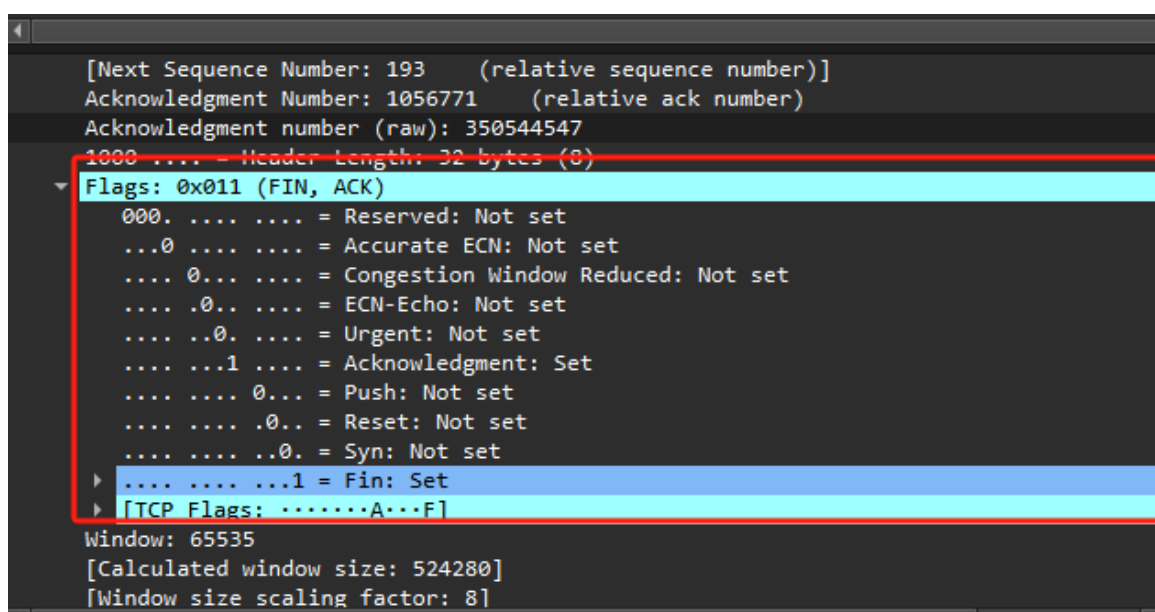
1. Packet 1 (Client → Server): Flags=[SYN], Seq = 0, Ack = 0。客户端发起连接请

求。

2. Packet 2 (Server → Client): Flags=[SYN, ACK], Seq = 0, Ack = 1。服务端回应, Ack 确认号为客户端 Seq+1。
3. Packet 3 (Client → Server): Flags=[ACK], Seq = 1, Ack = 1。客户端确认, 连接建立。

2.2 TCP 连接断开 (Teardown)

操作步骤: 观察 Trace 文件末尾的 FIN 或 RST 数据包。



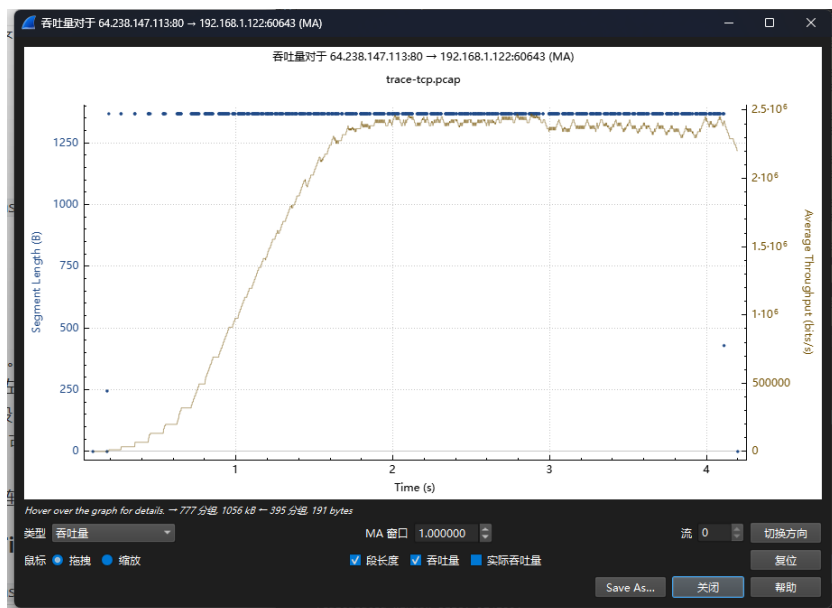
断开方式描述:

- 图片解读: 根据截图显示, 选中的数据包 Flags 字段为 0x011 (FIN, ACK)。
- 分析: 这表明连接是通过标准的四次挥手 (FIN Exchange) 来断开的。截图中的包由 Source (可能是本机) 发送, 序列号 (Seq) 为 193, 确认号 (Ack) 为 1056771。FIN 标志位的存在意味着发送方已经完成了数据发送任务, 希望优雅地关闭连接。

2.3 TCP 性能图表分析

2.3.1 TCP 吞吐量 (Throughput)

操作步骤: 菜单栏 Statistics → TCP Streams Graph → Throughput。

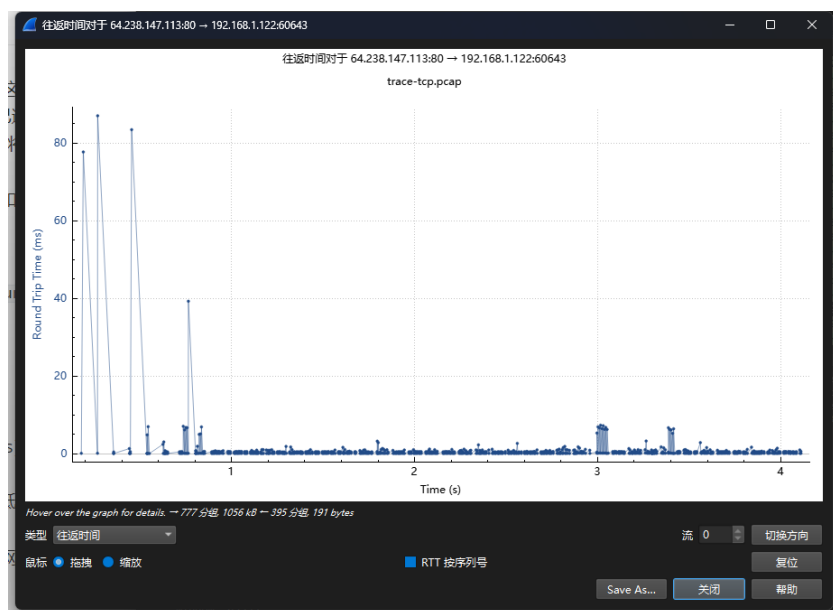


图表分析：

- 初始爬升 (0-1s)：图表左侧黄色线条从零开始稳步上升，这直观地展示了 TCP 的慢启动 (Slow Start) 阶段，发送窗口正在指数级扩张。
- 稳定吞吐 (1s-4s)：大约在 1 秒后，吞吐量稳定在 **2.5 Mbps** 左右（图中 y 轴刻度约为 2.5×10^6 ），且线条比较平滑。这表明连接进入了 **拥塞避免** 阶段，充分利用了当前链路的带宽。
- 结论：整个传输过程非常平稳，没有出现大幅度的锯齿状波动，说明网络状况良好，未发生严重的丢包或重传。

2.3.2 TCP 往返时间 (Round Trip Time)

操作步骤：菜单栏 Statistics → TCP Streams Graph → Round Trip Time。



图表分析：

- 初始峰值：如图表最左侧所示，连接建立初期（握手阶段）RTT 出现显著峰值（超过 80ms）。
- 稳定区域：握手完成后，RTT 迅速回落并稳定在极低的水平（大部分时间接近 0-10ms）。这在图中表现为底部的蓝色密集点。
- 小幅波动：在 x 轴约 3 秒处，出现了一些微小的 RTT 波动（上升），但这可能是短暂的网络排队延迟，并未影响整体的低延迟表现。

3 实验总结

通过本次实验，我结合 Wireshark 抓包工具深入理解了网络协议细节。在 UDP 部分，我通过观察具体的数据包（如 Port 53 的 DNS 包）验证了 UDP 头部 8 字节的简洁结构。在 TCP 部分，我不仅验证了三次握手的序列号变化，还通过观察 Flags 字段（如 FIN, ACK）识别了连接的断开方式。最后，通过吞吐量和 RTT 图表，我直观地评估了该 TCP 连接的稳定性与传输性能。